

# Apollo247 Selenium Pytest Automation Framework Documentation

---

## 1. Project Title

Apollo247 Healthcare Web Application Automation Testing using Selenium with Pytest Framework

## 2. Project Description

This project automates the Apollo247 healthcare web application using Selenium WebDriver with the Pytest framework. The framework follows the Page Object Model architecture and includes reusable utilities, logging, screenshots, and Allure reporting support.

- Pytest-based Selenium automation framework.
- Reusable Page Object Model design.
- Positive and negative scenario coverage.
- Integrated screenshot utility for failures.
- CSV and Excel test data integration.
- Allure reporting support.

## 3. Project Architecture

- tests/: Contains Pytest test cases.
- pages/: Contains Page Object Model classes.
- utils/: Utility classes for waits, logs, screenshots, and data readers.
- conftest.py: Common fixtures and WebDriver setup.
- pytest.ini: Pytest configurations.
- requirements.txt: Project dependencies.

## 4. Test Strategy

The project uses Pytest to execute automated UI functional testing scenarios with reusable fixtures and structured validations.

| Strategy Area    | Implementation                |
|------------------|-------------------------------|
| Framework Type   | Pytest                        |
| Design Pattern   | Page Object Model (POM)       |
| Automation Scope | Functional UI Testing         |
| Validation       | Assertions and UI validations |
| Execution        | Pytest command-line execution |
| Reporting        | Allure Reports                |
| Error Handling   | Screenshot capture and logs   |

## 5. Test Environment

| Component             | Details            |
|-----------------------|--------------------|
| Programming Language  | Python             |
| Automation Tool       | Selenium WebDriver |
| Test Framework        | Pytest             |
| IDE                   | PyCharm            |
| Browser               | Google Chrome      |
| Reporting Tool        | Allure Reports     |
| Operating System      | Windows            |
| Dependency Management | requirements.txt   |

## 6. Test Execution

The automation suite is executed through Pytest commands with Allure report integration.

- `pytest`
- `pytest -v`
- `pytest --alluredir=reports`
- `python main.py`

## 7. Test Scenarios Covered

- End-to-end healthcare workflow validation.
- Positive and negative login validation.
- Doctor search functionality testing.
- Doctor availability verification.
- Appointment booking flow validation.
- UI navigation and workflow testing.

## 8. Test Output

The framework generates detailed execution logs, screenshots, and report files after execution.

- Pytest execution summary.
- Passed and failed test statistics.
- Failure screenshots.
- Execution logs.
- Allure raw result files.

## 9. Test Report Analysis

Allure reporting enables detailed visualization and analysis of test execution results.

- Test case execution summary.
- Pass/fail analytics.
- Execution duration analysis.
- Failure diagnostics with screenshots.
- Improved debugging capability.
- Better maintainability and traceability.

## **10. Project Strengths**

- Reusable fixture-based architecture.
- Scalable automation design.
- Strong reporting integration.
- Efficient error handling.
- Maintainable POM structure.
- Supports regression testing execution.

## **11. Conclusion**

The Selenium Pytest automation framework provides a scalable and maintainable solution for validating Apollo247 application workflows. The framework improves test reliability, execution efficiency, and reporting visibility for automation testing activities.

# Allure reports screenshots

